TECHNICAL DEPLOYMENT RUNBOOK

StockRidge Multi-Client Deployment

A step-by-step guide for setting up, deploying, and maintaining StockRidge separately for multiple clients from one shared office Windows desktop.

AUDIENCE	Developers and deployment operators responsible for onboarding a new client onto StockRidge
ENVIRONMENT	One shared Windows laptop or desktop, Windows Terminal (PowerShell), VS Code
SYSTEMS IN SCOPE	Git and GitHub (multiple accounts and SSH identities), Cloudflare Workers, D1, Turso, the desktop till, and the Android app
DOCUMENT TYPE	Operational runbook — read in order for a new client, then keep for reference

Internal technical documentation — for the team responsible for client deployments. Revised 22 October 2026. Open dist\win-unpacked\StockRidge.exe. Do not open the portable exe.

About This Runbook

This runbook is for a developer or deployment operator using **one Windows laptop**, **VS Code**, **multiple GitHub accounts and SSH keys**, and **different Cloudflare accounts** for different StockRidge clients.

It assumes each client receives a separate folder, Git remote, Worker, database, custom domain and set of credentials. Never reuse one client's database, Worker name, Git remote, Cloudflare token, Turso token, or backup for another client.

CORE RULE

one client folder → one Git remote/SSH alias → one Cloudflare account → one Worker → one live database.

Everything below follows from that one rule. Every section either sets up one link in that chain, or verifies it before moving on. If a step ever produces a result that mixes two clients together, stop and fix it before continuing — see Section 21, “Stop Immediately If.”

StockRidge is multi-business inside one deployment. A client may run a bakery and a pharmacy as two businesses on their Worker. That is not a reason to put two clients on one Worker. Businesses share a database only when they belong to the same client.

ONE DATABASE, THREE APPS, AT THE SAME TIME

The demonstration PWA at `sample.stockridge.workers.dev`, the office host `stockridge.stockridge.workers.dev`, the desktop till, and the Android app all read and write one Turso database: `stockridge-10client.aws-eu-north-1.turso.io`. A sale on any of them is in that database once it has synced. The till's SQLite file is the offline queue, not a second ledger. The APK does not have its own cloud database. Do not create a second Turso database for the phone, the till, or the office host. Do not put the Turso token on a computer or a phone. A new client still gets their own database, and then that client's PWA, till, and APK all use that one database. Signing the same account in on a second device still signs the first device out. Different people can use the three apps together.

How to Use This Document

For a **brand-new client**, work through Sections 1–16 in order. Do not skip ahead. Later steps assume the folder, the SSH alias and the Cloudflare account are already the right ones. Section 17 is the counter desktop, when this client will use a Windows till rather than only the browser. Section 18 is the Android app, when a phone should open StockRidge as a program instead of only in Chrome. Sections 19–21 are for **ongoing operation** once a client is live: routine updates, quick checks, and the signs that mean stop.

THE CHECKOUT YOU ALREADY HAVE

This repository already contains a demonstration. The PWA at sample.stockridge.workers.dev and the office host stockridge.stockridge.workers.dev are the same Turso books, and the desktop till and Android app use those books at the same time. Update the demonstration with `--env=sample --skip-seed`, or the office host with `--skip-seed` and no environment flag, only when that host is the one you mean. Do not pass `--reset-pin`. Do not create a second database to separate them. Do not copy that Turso host or those Worker names onto a new client.

Prerequisites

Before starting on a new client, confirm the workstation and the access are ready:

- [ ] Windows laptop or desktop with administrator access (needed once, to enable the SSH agent service)
- [ ] Windows Terminal with PowerShell, and VS Code with the `code` command available on PATH
- [ ] Git for Windows installed (provides `ssh-keygen` and `ssh-add`)
- [ ] Node.js 20.11 or newer, and npm, so that `npm ci` and `node tools/deploy-cloudflare.js` run from the repository root
- [ ] Wrangler available to that script (`npx wrangler` is enough if it is not installed in `node_modules`)
- [ ] The Turso CLI, if this client will use Turso rather than D1 as the live database
- [ ] GitHub CLI (`gh`), only if the team prefers creating repositories from the terminal
- [ ] The correct GitHub account or organisation membership for this specific client
- [ ] The correct Cloudflare account access for this specific client
- [ ] A secure place for the client deployment register — see Section 2. Identifiers only. No tokens, no PIN.
- [ ] The StockRidge source package, ready to extract into the new client's folder
- [ ] The desktop shell, when this client will have a counter program. Clone it beside that client's checkout in Section 17. It is not a second Worker, and it does not get a database token.

Key Terms

SSH alias	A short nickname (for example `github-northline`) in `~\.ssh\config` that tells Git which SSH key to present to github.com, so several GitHub identities can coexist on one machine.
Worker	The Cloudflare Worker that runs StockRidge's API and serves the client's app from the same hostname. There is no second backend and no separate Pages site.
D1 database	Cloudflare's SQL database attached to a Worker. The binding stays in `worker/wrangler.toml` so Wrangler can deploy. It is the live store only when Turso is not configured.
Turso database	The live SQL database for a hosted client. The URL is not a secret. The auth token is. Sample uses Turso because a Worker on the Free plan cannot raise its subrequest cap, and one Turso socket keeps a shop screen inside that cap.
StockRidge Admin	The one platform administrator. No business, no branch. Created by the seed, used to sign in and create the client's proprietor. Not the person who rings sales.
Proprietor	The client's owner. Stored as role `OWNER` with no business on the row. Owns every business and branch of that deployment, including ones opened later. Created in the app, not in SQL.
Environment	A named block in `worker/wrangler.toml`. `--env=sample` deploys sample. Omitting `--env` deploys the top-level configuration, which is production.
Desktop till	A standalone window for one shop computer. The screens travel inside the program. A file shipped in that program wins over a copy downloaded from the Worker, so a Worker deploy does not update the till. API calls go to this client's Worker. Offline books on that computer are a SQLite file. The till does not hold a Turso token, and it does not sync to Turso itself.

Table of Contents

01 Set Up a Separate Folder for Every Client

02 Keep a Client Deployment Register

03 Configure One SSH Key per GitHub Account

04 Create SSH Aliases in ~\.ssh\config

05 Create the New Client Repository with the Correct SSH Identity

06 GitHub CLI Is Separate from Git SSH

07 Install Dependencies and Connect Wrangler

08 Choose Names — and Do Not Reuse the Downloaded Ones

09 Point wrangler.toml at the New Worker and Database

10 Put Credentials in .env.deploy

11 Apply Migrations to the Database the Worker Will Read

12 Create the First Administrator, and Treat the PIN as a Secret

13 Deploy the Worker and the PWA

14 Create the Proprietor and the First Business

15 Optional: Attach a Client-Owned Custom Domain

16 First Production Smoke Test

17 Put a Standalone Till on the Shop Computer

18 Put StockRidge on an Android Phone

19 Day-Two Deploy and Update Routine

20 Useful Checks

21 Stop Immediately If Any of These Happen

01 Set Up a Separate Folder for Every Client

Every client's copy of StockRidge lives in its own folder. Mixing clients into one folder — and repeatedly editing `worker\wrangler.toml` and `.env.deploy` to "switch" between them — is the usual way a deploy writes to the wrong database.

Create a stable root folder once:

```
New-Item -ItemType Directory -Force "C:\StockRidgeClients"
Set-Location "C:\StockRidgeClients"
```

Recommended layout — one sub-folder per client, each containing its own full copy of the project:

```
C:\StockRidgeClients\
├── NorthlineStores\
│       ├── stockridge\
│       └── destop-stockridge\
├── HarbourMart\
│       ├── stockridge\
│       └── destop-stockridge\
└── SampleDemonstration\
        ├── stockridge\
        └── destop-stockridge\
```

`destop-stockridge` is the counter program. Add it in Section 17, beside that client's checkout, only when the shop will use a desktop till. It is not a second copy of the Worker, and it is not a place for this client's tokens.

DO NOT

Keep several clients in one mutable folder and change `worker\wrangler.toml` or `.env.deploy` to switch between them. That is how a deploy lands in the wrong D1 database, or points a new Worker at the sample Turso host.

For a new client folder:

```
$Client = "NorthlineStores"
$ClientRoot = "C:\StockRidgeClients\$Client\stockridge"
New-Item -ItemType Directory -Force $ClientRoot
Set-Location $ClientRoot
```

Copy or extract the StockRidge source into `$ClientRoot`, then open that folder in VS Code:

```
code .
```

02 Keep a Client Deployment Register

One row per client, kept somewhere you will actually open before every deploy. This is the source of truth for which folder, SSH alias, Cloudflare account, Worker and database belong together.

> **NOTE**
> Create this record **outside Git** — a password-manager secure note, or an encrypted company record. Store identifiers only. Do not store the Cloudflare token, the Turso token, the JWT secret, or the administrator PIN.

CLIENT	LOCAL FOLDER	SSH ALIAS	GITHUB REPO
Northline Stores	C:\StockRidgeClients\NorthlineStores\stockridge	github-northline	northline-org/stockridge

CLOUDFLARE ACCOUNT	WORKER	LIVE DATABASE	DOMAIN
stored securely, not in Git	northline-stockridge	northline-stockridge (Turso) — not the sample host	app.northline.example

Before every deploy, read the current row and confirm the folder, the remote, the Worker, the environment flag and the database all agree. If the Turso host in the register is `stockridge-10client.aws-eu-north-1.turso.io`, you are looking at the demonstration, not at a new client.

When this client has a desktop till, add two more fields to the same row: the Worker URL baked into the archive, and the renamed file you handed the shop. Those two must name this client. The desktop folder is a clone of the shared shell. It does not get its own Turso database.

03 Configure One SSH Key per GitHub Account

Each client's repository lives under a specific GitHub account or organisation. Git needs a dedicated SSH key for that account so it never authenticates as the wrong identity.

3.1 Check existing SSH keys

```
Get-ChildItem "$HOME\.ssh"
```

> **CAUTION**
> If a key already belongs to another GitHub account, do not overwrite it.

3.2 Create a new client GitHub key, if needed

Name the key after the **GitHub account**, not after the database:

```
$GitHubKey = "$HOME\.ssh\id_ed25519_github_northline"
ssh-keygen -t ed25519 -a 64 -C "github-northline-deployment" -f $GitHubKey
```

Choose a passphrase. Do not use the administrator PIN, or the proprietor's PIN, as an SSH passphrase.

Copy the public key and add it to the intended GitHub account under Settings → SSH and GPG keys:

```
Get-Content "$GitHubKey.pub" | Set-Clipboard
Start-Process "https://github.com/settings/keys"
```

If the repository belongs to an organisation using SSO, authorise the new key for that organisation in GitHub.

3.3 Start the Windows SSH agent and load the key

Run this once from an **Administrator** PowerShell, only if the service is disabled:

```
Set-Service -Name ssh-agent -StartupType Automatic
Start-Service ssh-agent
```

Then, from the normal VS Code terminal:

```
ssh-add $GitHubKey
ssh-add -l
```

04 Create SSH Aliases in ~\.ssh\config

An alias stops Git from guessing which key to use for `github.com`. It maps a short host name onto one identity file.

```
$SshConfig = "$HOME\.ssh\config"
$KeyForConfig = ($GitHubKey -replace '\\','/')
@"
Host github-northline
    HostName github.com
    User git
    IdentityFile $KeyForConfig
    IdentitiesOnly yes
"@ | Add-Content $SshConfig
```

Example with two GitHub accounts side by side:

```
Host github-personal
    HostName github.com
    User git
    IdentityFile C:/Users/YourName/.ssh/id_ed25519_personal
    IdentitiesOnly yes

Host github-northline
    HostName github.com
    User git
    IdentityFile C:/Users/YourName/.ssh/id_ed25519_github_northline
    IdentitiesOnly yes
```

Test the intended identity:

```
ssh -T git@github-northline
```

EXPECTED

GitHub replies with a successful authentication message followed by “does not provide shell access.” That is expected. It is not an error.

DO NOT

Use `git@github.com:...` for a multi-account client repository. Use the alias: `git@github-northline:northline-org/stockridge.git`.

05 Create the New Client Repository with the Correct SSH Identity

From the new client folder:

```
Set-Location "C:\StockRidgeClients\NorthlineStores\stockridge"
```

If the downloaded folder contains the source project's old Git history and you want a new, clean repository:

```
if (Test-Path .git) {
    Remove-Item -Recurse -Force .git
}
git init
git branch -M main
git config --local user.name "Your Northline GitHub identity"
git config --local user.email "your-northline-github-email@example.com"
git add .
git commit -m "Initial Northline Stores StockRidge deployment"
```

NOTE

Set the name and email with `--local`, inside this folder only. A global identity becomes the wrong author on the next client's commit.

Create an empty **private** repository in the intended GitHub account or organisation. Then connect it with the SSH alias:

```
git remote add origin "git@github-northline:northline-org/stockridge.git"
git push -u origin main
```

Verify the boundaries before continuing:

```
git remote -v
git config --local user.name
git config --local user.email
git status
```

EXPECTED REMOTE

`git@github-northline:northline-org/stockridge.git`

If it says `github.com` without the alias, correct it before the next push:

```
git remote set-url origin "git@github-northline:northline-org/stockridge.git"
```

DO NOT

Commit `.env.deploy`, `.env`, `.data\d1-seed.sql`, a database file, a backup, or a token. Those paths are gitignored. `git status` showing one of them means the ignore failed — do not add it.

06 GitHub CLI Is Separate from Git SSH

Git uses the SSH alias to push and pull. The GitHub CLI (`gh`) is independent. It uses its own active browser or token session. You can create the repository in the browser and avoid switching CLI accounts entirely.

If you do use `gh`, check the active account before creating a repository:

```
gh auth status
```

Sign in to the intended GitHub account when required:

```
gh auth login --hostname github.com --git-protocol ssh --web
gh auth status
```

Use `gh auth switch` only after checking which account it selects:

```
gh auth switch --hostname github.com
```

DO NOT

Paste a GitHub personal access token into a file that Git tracks. A token in `.env.deploy` stays on the machine. It does not go in the repository.

07 Install Dependencies and Connect Wrangler

StockRidge deploys from the repository root, not from a hand-typed list of Wrangler commands. The script is `tools\deploy-cloudflare.js`. Install the locked dependency set first.

```
Set-Location "C:\StockRidgeClients\NorthlineStores\stockridge"
npm ci
```

NOTE

Use `npm ci`, not `npm audit fix --force`. The lock file is the set this project was tested with. `package.json` is at the root. Do not run `npm ci` inside `worker` unless that folder has its own lock file — it does not.

Confirm Wrangler can see **this client's** Cloudflare account. The browser window that opens during login is normal.

```
npx wrangler login
npx wrangler whoami
```

DO NOT

Paste a Cloudflare API token into a Git-tracked file, into `worker\wrangler.toml`, or into a screenshot that will be committed. The token belongs in `.env.deploy`, which is gitignored. See Section 10.

08 Choose Names — and Do Not Reuse the Downloaded Ones

Use lowercase letters, numbers and hyphens. The downloaded `worker\wrangler.toml` already names a demonstration. Those names and IDs belong to that demonstration. A new client does not inherit them.

```
$WorkerName = "northline-stockridge"
$DatabaseName = "northline-stockridge-db"
$TursoName = "northline-stockridge"
```

DO NOT REUSE THESE DOWNLOADED VALUES

Worker `stockridge` or `sample`. D1 database `stockridge` (`32aa519c-a7fb-41d5-bc5b-083d0a0489bc`). D1 database `stockridge-staging` (`abf164d9-f3bb-4e56-9a9f-addd795013f7`). D1 database `stockridge-sample` (`fd72e95b-c0c8-4073-8aba-d3ca5919b107`). Turso host `stockridge-10client.aws-eu-north-1.turso.io`.

Which database this client will actually read

Pick one live store. Do not point the Worker at both and hope.

PATH	USE IT WHEN
Turso	A hosted shop. Sample and the office Worker both use `stockridge-10client.aws-eu-north-1.turso.io`. The Worker opens one TLS socket to Turso, so a screen is not cut off at the Free plan's 50 external subrequests. `sample` and `production` refuse to fall back to D1.
D1	Only when Turso is not configured, and the environment is not `sample` or `production`. The office host no longer reads D1. Do not copy that old D1 database onto Turso.

Create the new D1 database in this client's Cloudflare account. The binding is required even when Turso is the live store, because Wrangler deploys against it. The Worker does not query that D1 once a Turso URL and token are both present.

```
npx wrangler d1 create $DatabaseName
```

EXPECTED

Cloudflare prints a new `database_id`. Copy that id. It must not be one of the three ids in the box above.

```
$DatabaseId = Read-Host "Paste the new Cloudflare D1 database_id"
```

Create the Turso database, if Turso is the live store

Do not create a token for the sample database.

```
turso db create $TursoName
turso db show $TursoName --url
turso db tokens create $TursoName
```

The URL may come back as `libsql://...`. In `wrangler.toml`, write the `https://` form of the same host. The token is shown once by the Turso CLI. Put it in `.env.deploy` and nowhere else. Do not paste it into a commit message.

09 Point wrangler.toml at the New Worker and Database

The file is `worker\wrangler.toml`, not `wrangler.jsonc`. A new client should not keep the downloaded top-level name `stockridge` if that Worker already exists on another account — and must not keep it if this Cloudflare account already has the demonstration Worker.

From the repository root, replace the Worker name, the D1 name and the D1 id in the block you are about to deploy. The script later rewrites `database_id` for the environment it is deploying, and only that block. It does not rewrite the other environments. That is why the folder must already be this client's folder before you run it.

For a client whose production is the top-level block, the values that must change are:

```
name = "northline-stockridge"

[[d1_databases]]
binding = "DB"
database_name = "northline-stockridge-db"
database_id = "<the new id from Section 8>"
migrations_dir = "../schema/migrations"

[vars]
ENVIRONMENT = "production"
DB_NAME = "northline-stockridge-db"
```

If this client uses Turso, add the URL to that same `[vars]` block. Do not add the token here.

```
TURSO_DATABASE_URL = "https://northline-stockridge-<org>.turso.io"
```

ENVIRONMENTS

`--env=sample` reads `[env.sample]`. Omitting `--env` reads the top of the file, which is production. `--env=prod` is not a declared environment. The script refuses an unknown name. A missing flag is not refused — it deploys production. `--env=production` is the named production block. In the downloaded file that block and the top level point at the same demonstration database. On a new client, make them agree with the register, or delete the environment blocks this client will not use.

Review the names before the first deploy:

```
Select-String -Path .\worker\wrangler.toml -Pattern "name =|database_name|database_id"
Select-String -Path .\worker\wrangler.toml -Pattern "TURSO_DATABASE_URL|ENVIRONMENT"
```

EXPECTED

You see the new Worker name, the new D1 name, the new D1 id, and — if Turso is the live store — a Turso host that is not `stockridge-10client`. You do not see a token.

A D1 database id is not a password. The Turso hostname is not a password. Keeping the client repository private is still the right default. Commit the non-secret configuration after you have reviewed it:

```
git add .\worker\wrangler.toml
git commit -m "Configure Northline Worker and database"
git push
```

10 Put Credentials in .env.deploy

The deploy script reads credentials from the environment, or from `.env.deploy` at the repository root. That file is gitignored. One client's file must not be copied into another client's folder.

```
# .env.deploy — this client only. Never commit.
CLOUDFLARE_API_TOKEN=...
CLOUDFLARE_ACCOUNT_ID=...
STOCKRIDGE_ADMIN_USERNAME=admin
STOCKRIDGE_JWT_SECRET=...
```

Add the Turso lines only when this client uses Turso, and only for the database created in Section 8:

```
TURSO_DATABASE_URL=https://northline-stockridge-<org>.turso.io
TURSO_AUTH_TOKEN=...
```

Generate the JWT secret in the terminal. Do not invent a short one, and do not reuse another client's.

```
$Jwt = node -e "console.log(require('crypto').randomBytes(48).toString('base64url'))"
# Put $Jwt into .env.deploy as STOCKRIDGE_JWT_SECRET, then:
Remove-Variable Jwt
```

NEVER PLACE A PRODUCTION SECRET IN

`worker\wrangler.toml` · a file Git tracks · a commit message · a PowerShell script that is committed · a screenshot · the deployment register

The script writes `STOCKRIDGE_JWT_SECRET` to the Worker as the secret `JWT_SECRET`. If that variable is missing on a later deploy, the existing Worker secret is left alone, so a shop day is not signed out. Set it before the first deploy, and keep the same value for later deploys of this client.

Confirm Git will not pick the file up:

```
git check-ignore .env.deploy
git status
```

EXPECTED

`git check-ignore` prints `.env.deploy`. `git status` does not list it as a file to commit.

11 Apply Migrations to the Database the Worker Will Read

The schema is the SQL files in `schema\migrations`. There is one set. D1 records applied files in `d1_migrations`. Turso records them in `_migrations`. Do not edit a file that has already been applied. Do not recreate a deleted `0023`. A change is a new file.

Turso path

With this client's URL and token in `.env.deploy`, from the repository root:

```
node tools/turso-admin.js migrate
```

EXPECTED

A JSON line listing the migrations applied. A second run applies none and reports the ones already recorded. The token is not printed.

DO NOT

Run `node tools/turso-admin.js copy-from-d1` as part of onboarding, or on any later deploy. That copy is a one-time move off an old D1 database. Running it again overwrites rows the shop has written on Turso. Do not copy the demonstration database into a new client.

Migration `0013` loads the NAFDAC Greenbook as reference data — just under 6,800 rows. A count of zero means that migration did not apply. A non-zero count does not mean this shop is a pharmacy. The Greenbook is used only when the business vertical is pharmaceutical.

D1 path

Leave `TURSO_DATABASE_URL` and `TURSO_AUTH_TOKEN` unset. Do not name the environment `sample`. The deploy script in Section 13 applies the same SQL files to the remote D1 database. You do not run a separate migrate command first.

CAUTION

If both Turso variables are set, the deploy script skips D1 migrations on purpose and applies schema to Turso instead. A half-configured file — a URL without a token, or the sample URL copied by mistake — is a reason to stop, not a reason to continue.

12 Create the First Administrator, and Treat the PIN as a Secret

A migrated database has the schema. It does not yet have a person. The seed creates exactly three things: the settings row, the withholding-tax schedule as editable rows, and one `ADMIN` account with no business and no branch. It does not create a business, a proprietor, stock, or a sample PIN for staff.

The PIN is digits only, at least four. The deploy script can generate a five-digit PIN. Only a hash is stored. The PIN is shown once. It is not written into Git.

D1 is the live store

Do not seed by hand. The first deploy in Section 13 creates the administrator. Read the PIN from the summary only when the summary says the PIN is in effect. A line that says the PIN is unverified means the script could not read the administrator count. That printed number is not the live PIN. Do not hand it to anyone.

A later deploy must pass `--skip-seed`. The seed is `INSERT OR IGNORE`, so a normal re-run would not change an existing PIN, but the script may still print a freshly generated number that does not work.

Turso is the live store

The deploy script's seed still writes to the D1 binding. The Worker will not read that D1 once Turso is configured. Seeding D1 does not create the person who can sign in. Generate the SQL yourself, apply it to this Turso database, then delete the file.

```
$Pin = Read-Host "Choose the administrator PIN (digits, at least 4)"
node tools/d1-seed.js --out=.data/d1-seed.sql --username=admin --pin=$Pin
cmd /c "turso db shell northline-stockridge < .data\d1-seed.sql"
Remove-Item .\.data\d1-seed.sql
Remove-Variable Pin
```

Confirm the account without printing a hash:

```
$Sql = "SELECT username, role, is_active FROM users WHERE role = 'ADMIN'"
turso db shell northline-stockridge $Sql
```

EXPECTED

One row: username `admin`, role `ADMIN`, active. Then deploy with `--skip-seed`, so the script does not print a different PIN from the unused D1 database.

DO NOT

Pass `--reset-pin` unless the administrator PIN is lost and you intend to overwrite it. That flag rewrites the stored hash. Do not create the proprietor, a co-owner, or a cashier in SQL. Do not store the PIN in `.env.deploy`. Do not re-hash the PIN with another tool: a hash written above 100,000 PBKDF2 iterations cannot be checked by the Worker.

13 Deploy the Worker and the PWA

One command stamps the build, checks the Cloudflare token, prepares the database, sets secrets, uploads the Worker, and smoke-tests the URL. The Worker serves both `/api/*` and the app. Do not deploy `public\` to Pages as a second site.

The stamp is written for you into `public\js\app.js`, `public\sw.js` and the asset URLs. Its clock is West Africa Time. Do not edit `SR.BUILD` by hand. A browser that keeps the previous stamp keeps the previous screens.

INTENT	COMMAND, FROM THIS CLIENT'S FOLDER
First deploy, D1 is the live store	`node tools/deploy-cloudflare.js`
First deploy, Turso already seeded	`node tools/deploy-cloudflare.js --skip-seed`
Update the existing demonstration	`node tools/deploy-cloudflare.js --env=sample --skip-seed`
Later update of this client	the same command as the first deploy, plus `--skip-seed`

DO NOT

Pass `--reset-pin`. Do not omit `--env` when you meant sample. Do not treat `--dry-run` as a safe preview: it skips the Worker upload only. It still stamps files and still prepares the database named in the config.

Read the summary before you close the terminal.

- Environment is the one in the register. “production (default)” means you omitted `--env`.
- Other environments are listed as untouched. If one of them points at this database, stop.
- The Worker URL is the one you will smoke-test.
- Turso, if used, is this client's host. The token line says it was stored and not printed.
- The PIN line, on a D1 first deploy, is in effect only when the summary says so. Unverified means ignore the number.
- Readiness `awaiting_first_business` is the correct handover state. It is not a failed deploy.

EXPECTED HEALTH

`/api/health` returns JSON with `"ok": true`, `"status": "up"` and `"service": "stockridge"`. HTML means the asset store answered, or another application owns this Worker name.

```
$WorkerUrl = "https://northline-stockridge.<account>.workers.dev"
Invoke-RestMethod "$WorkerUrl/api/health"
Start-Process $WorkerUrl
```

After a successful deploy, commit the stamp files the script changed — `public\index.html`, `public\js\app.js`, `public\sw.js`, and the two route files if they changed — so the private repository matches what the browser loads. Do not commit `.env.deploy` in that commit.

14 Create the Proprietor and the First Business

Sign in with the administrator from Section 12. Open **Businesses** and create the client's first business. Choose the vertical that matches the shop. Provisioning builds the chart of accounts, categories, price lists and a starter catalogue, and it creates the first branch with the business. That is the audited start of the client's books. Do not insert those rows in SQL.

Then open **Staff** and create the proprietor: role Owner, with no business pinned on the account. The proprietor owns every business and every branch on this deployment, including one opened later. Hand them their own PIN in private. Ask them to sign in and confirm they can see the business.

DO NOT

Create, publish, or reuse a sample PIN. Do not create a fifth stored role. Co-owner is `OWNER` with a business on the row, plus the businesses they are granted — not a separate role, and not a job title. A general manager is `MANAGER` with no branch, and covers every branch of their business, not every business on the deployment. A branch manager and a member of staff are pinned to one branch.

Verticals the create-business screen can provision

CODE	WHAT IT IS
ELECTRONICS	Electronics, appliances and gadgets
FURNITURE	Furniture and home furnishings
BUILDING_MATERIALS	Building materials and hardware
WHOLESALE_RETAIL	Wholesale and retail general merchandise
SUPERMARKET	Supermarket
PROVISIONS_FABRICS	Provisions and fabrics
GENERAL_RETAIL	General retail, fully configurable
PHARMACEUTICAL	Dispensary. Greenbook and dispensing belong here, not on the other rows.
WATER_PACKING	Sachet and bottled water. Packaged water, not a pharmacy.
BAKERY	Bakery, retail and wholesale. Not a pharmacy.
EVENT_RENTAL	Event rental. The branch type is rental.
EQUIPMENT_RENTAL	Equipment rental
LEASING	Leasing
SELL_AND_LEASE	Sell and lease. Do not turn this into a rental yard by default.

A client with two trades gets two businesses on this same Worker, each with its own vertical. They do not get a second Worker unless they are a second client.

When a branch is created, give it a real code. A blank code must not be saved as `MAIN`. The code appears on transfer references.

15 Optional: Attach a Client-Owned Custom Domain

The domain must already be on the same Cloudflare account as the Worker. For a dedicated hostname, add a custom-domain route to the block you deploy. Example, on the top-level production block:

```
[[routes]]
pattern = "app.northline.example"
custom_domain = true
```

For a named environment, the header is `[[env.production.routes]]` or `[[env.sample.routes]]`, matching the block. Keep `run_worker_first = ["/api/*"]` in the assets block. That is what sends API calls to the Worker instead of the asset store.

Deploy again with the same command you used in Section 13, including `--skip-seed` now that the administrator exists.

```
$AppUrl = "https://app.northline.example"
Invoke-RestMethod "$AppUrl/api/health"
Start-Process $AppUrl
```

NOTE

This deployment serves the whole app and `/api/*` from the same hostname. Do not separately deploy `public\` to Pages unless you intentionally redesign routing. A health URL that returns HTML is the symptom of that mistake.

16 First Production Smoke Test

Complete these in the browser before handing the app to the client's team. Use the client's URL, not the demonstration URL, unless you are checking the demonstration on purpose.

- [ ] `/api/health` returns JSON, not HTML. `/api/health/ready` is `awaiting_first_business` before the first business, and `ready` after it.
- [ ] Administrator signs in with the PIN from Section 12. A PIN printed as unverified does not count.
- [ ] Administrator creates one real business and chooses the vertical. A branch is created with it. The branch has a real code.
- [ ] Administrator creates the proprietor. The proprietor signs in on their own PIN and can see that business.
- [ ] Proprietor creates one general manager, or one branch manager, and one member of staff. A general manager with more than one shop can open the top-bar shop control and pick a shop. It must not stay a blank dash.
- [ ] Manager creates a supplier and receives a small test batch into the shop that was selected, not into “all shops”.
- [ ] Staff opens a till, completes one cash sale, and reviews the receipt. Cancel sits beside Print. The receipt does not invent a second page of system codes.
- [ ] Manager closes the till against the physical expected amount.

- [ ] Proprietor checks Dashboard, Accounting, Staff, Branches, and Settings. Settings save on their own tab. Branding is not mixed into a settings save that the screen did not offer.
- [ ] Signing the same account in on a second device signs the first device out of the app. That is the rule, not a failed session.
- [ ] Install the app on the counter phone or laptop. If a launcher icon was cached from an older build, remove it and install again, then confirm the build stamp in the app matches the deploy summary.

DO NOT

Run `node tools/format-admin-only.js` to “clean up” a smoke test on a database that already has real sales. That tool deletes businesses and trading rows and leaves the administrator. It refuses to run without `--yes` and without `--env=sample` or `--env=production`. It is not a reset button for a live client.

17 Put a Standalone Till on the Shop Computer

Do this only when the counter will run StockRidge as a program, not only as a browser tab. A phone still installs the browser app from the Worker URL. The desktop till is a window around the same screens. It is not a second deployment, and it is not a second database in the cloud. The PWA and the till use one Worker. That Worker is the door into Turso. Water packing, bakery and the other verticals are on that Worker. They are not a separate Electron list.

Offline sales on that computer are written to a SQLite file before the cashier is told the sale succeeded. When the link returns, the existing sync posts that queue to this client's Worker, then pulls. The till never opens Turso itself. Do not put a Turso token, a Cloudflare token, or a PIN in the desktop folder. The file is SQLCipher. The key stays in the Windows credential store. Do not delete `stockridge.key`. The window may copy screens from the Worker into a cache. That cache does not replace a file already inside the program. A Worker deploy updates the browser. It does not update the till. A screen fix reaches the till only when you stage that client's `stockridge\public` and build the unpacked program again. The office host `stockridge.stockridge.workers.dev`, the PWA at `sample.stockridge.workers.dev`, this till, and the Android app read one Turso database at the same time: `stockridge-10client.aws-eu-north-1.turso.io`. Water packing, bakery and the other verticals are on that database. The old D1 database was not copied back, and no PIN was changed. Do not create a second database so the phone can have its own books.

Sign in once while the internet works. After that, the same PIN opens the saved session when the Worker cannot be reached. A person who has never signed in on that computer still needs the internet once. One counter can start the LAN hub from the File menu and share the last mirror plus queued quantities. Other tills set `STOCKRIDGE_HUB` and `STOCKRIDGE_HUB_SECRET`. The hub does not accept a sale in place of the Worker. Code signing needs `CSC_LINK` and `CSC_KEY_PASSWORD`. Without that certificate the program is unsigned. Updates stay off until `STOCKRIDGE_UPDATE_URL` is set.

17.1 Clone the shell beside this client's checkout

The shell is one shared repository. Each client still gets their own clone, next to that client's `stockridge` folder, so the build copies that client's screens and not another shop's. Pull each repository in its own folder. Do not run `git pull` in the parent folder. A parent path with a space, such as `C:\Users\Mohammed Hayatudeen\StockRidge`, is split by `npm.cmd`, and the build never runs.

```
Set-Location "C:\StockRidgeClients\NorthlineStores"
git clone "https://github.com/10client/destop-stockridge.git" destop-stockridge

Set-Location "C:\StockRidgeClients\NorthlineStores\stockridge"
git pull
git rev-parse --short HEAD

Set-Location "C:\StockRidgeClients\NorthlineStores\destop-stockridge"
git pull
git rev-parse --short HEAD
npm install
```

NOTE

If the desktop folder is not cloned yet, clone it into `destop-stockridge` beside `stockridge`, then pull. If GitHub asks for a login, use the office account that can read that repository. Do not point this folder at the client's stockridge remote. Do not copy `.env.deploy` into it. Write both short hashes in the register before you build. Those hashes are the program you are about to hand over. As of 22 October 2026 the demonstration till fixes are destop `2c13aed` and stockridge `bf3f961`. A later hash in the same folder replaces them only after you print it and build again.

17.2 Build the unpacked program for this client only

Deploy the Worker first (Section 13) so this client has a host to sign in against. The screens inside the program come from the sibling `stockridge\public` at the moment you stage, not from the live Worker. Set the origin, stage, then build. Do not start the build through `npm.cmd` when the folder path contains a space.

```
Set-Location "C:\StockRidgeClients\NorthlineStores\destop-stockridge"
$env:STOCKRIDGE_ORIGIN = "https://northline-stockridge.<account>.workers.dev"
node .\tools\stage-ui.js
node .\node_modules\electron-builder\cli.js --win dir --config.win.signAndEditExecutable=false
Remove-Item Env:STOCKRIDGE_ORIGIN
Get-Content .\build\desktop-config.json
```

EXPECTED

The stage line prints `This archive will sign in against` this client's Worker URL. `desktop-config.json` contains that same URL and no token. The program you open is `dist\win-unpacked\StockRidge.exe`.

DO NOT

Leave `STOCKRIDGE_ORIGIN` unset when this is a client. The unset default is the demonstration, `sample.stockridge.workers.dev`. Do not leave the variable set and then build the next client. The office host is the same Turso books as the demonstration. A client must not be built against either of those two URLs. Do not run `npm run archive` from a parent folder. Do not spawn electron-builder through `shell: true`.

If the native SQLite step fails, install the Visual Studio C++ build tools and Python, then run the same two `node` commands again with the same origin. Do not hand over a half-built `dist` folder.

The Windows file is not code-signed until `CSC_LINK` exists. Leave `signAndEditExecutable` false. Do not turn signing on for this handover.

17.3 Copy the unpacked folder, not the portable exe

`dist\win-unpacked\StockRidge.exe` is the launcher inside a folder. It runs the files beside it. That folder is what a new build replaces. Copy the whole `win-unpacked` folder to the shop computer, and rename the folder with the client name before it can be mixed up, for example `StockRidge-Northline-win-unpacked`. The shop computer does not need Node, Git, or the checkout.

`dist\StockRidge-0.1.0-win-x64.exe` is an old single-file portable package. The current build does not write it. The copy left on the office computer is the locked file from 20 October 2026, 4:37:44 PM, length 77,304,820. Opening it runs that old program. A build that tries to overwrite it waits until it is cancelled, so the new fixes never land in it. Do not open it. Do not rename it and hand it over. Do not tell the shop to use it.

The first launch creates `stockridge.db` in the Windows app-data folder of the user who is signed in. File → Save a copy of this till is the backup of sales that have not synced yet. Keep that backup with this client's records, not in Git.

> **NOTE**
> This Windows laptop builds a Windows program. A Mac counter needs the archive built on a Mac. Do not send the `.exe` to a Mac and expect it to open.

17.4 Open it once before you leave

The window is at `127.0.0.1:47321` on that computer. Sign in with a staff PIN for this client, not with a sample PIN. Complete one cash sale, or confirm an already synced till opens. A second window is refused, so two copies cannot write the same file. If the port is already in use, close the other StockRidge window rather than building a second archive.

A till that has already been selling in Chrome does not hand its queue over by itself. On that browser, open Sync and choose Back up this device. In the desktop window, open Sync and choose Restore from a backup. Do that before abandoning the browser, or unsynced sales stay in Chrome.

The window must not show Download or Install. Those buttons belong on the browser, including a home-screen app. The program removes them, even when the hosted screens still contain them. If Install is still in the sidebar, this computer is running an older program, often the portable exe. Close it. Pull both repositories in their own folders, print the short hashes, and build the unpacked program again. Do not deploy the website just to hide that button.

> **DO NOT**
> Treat `npm start` as the handover. That is the development window. It still needs the checkout beside it, and with no origin set it signs in against the sample Worker. Do not point the till at Turso Sync. Push, then pull, through this client's Worker, is the only door into the cloud database.

18 Put StockRidge on an Android Phone

Do this only when a phone should open StockRidge as a program, not only as a Chrome tab. Chrome on that phone still shows Download and Install. That is the PWA, and it stays. The APK is a

WebView around this client's Worker. It is the same screens and the same books. It removes Download and Install itself, so a website deploy is not required for that. The APK does not hold a Turso token, a Cloudflare token, or a PIN, and it does not open Turso.

The project is `destop-stockridge\android`, in the same shell repository as the Windows till. It is not a second Worker and not a second database. For the demonstration, leave the origin as `https://sample.stockridge.workers.dev`. That is the same Turso database as the office host, the PWA, and the desktop till. Open the folder in Android Studio on a machine that has the Android SDK. Before you build a different client's copy, set the origin in `app\src\main\res\values\strings.xml` to that client's Worker URL, the same URL you baked into that client's desktop archive.

```
Set-Location "C:\StockRidgeClients\NorthlineStores\destop-stockridge"
git pull
```

In Android Studio: File → Open, and choose the `android` folder. Wait for Gradle. Then Build → Build Bundle(s) / APK(s) → Build APK(s). The file is `app\build\outputs\apk\debug\app-debug.apk`. Rename it with the client name before it can be mixed up. A release build for a shop needs a signing key that stays with this client. Do not commit that key, and do not commit the APK.

EXPECTED

The installed app opens the same Worker as that client's PWA and desktop till. A sale on the phone is in the same database as a sale on the till, once it has synced. The top bar and the sidebar do not offer Download or Install. Chrome on the same phone, opened to the same URL, may still offer them.

DO NOT

Point a different client's APK at the demonstration or at the office host. Those two hosts are the same database, and they are not that client's database. Do not create a second Turso database for the APK. Do not put a Turso token in the Android project. Do not expect Chrome on the phone to hide Install. Chrome is the PWA. Do not deploy the website just to hide the button inside the APK.

19 Day-Two Deploy and Update Routine

From the VS Code terminal at **this client's** repository root:

```
git pull
npm ci
node tools/deploy-cloudflare.js --skip-seed
git status
```

Add `--env=sample` only when the register says this folder's target is the demonstration. Add `--env=<name>` only when the register names that environment. Then:

pull source → install locked dependencies → deploy (the script applies new migrations to the live store) → read the summary → smoke-test `/api/health` and one signed-in screen → commit the stamp files if they changed.

DO NOT

Run a destructive reset against a real client's database. Do not pass `--reset-pin`. Do not run `copy-from-d1`. Do not omit `--skip-seed` on a database that already has an administrator. Take a backup and confirm the target database name before any remote SQL you type by hand.

UPDATING THE DEMONSTRATION THAT ALREADY EXISTS

From the demonstration folder only: `node tools/deploy-cloudflare.js --env=sample --skip-seed`. Production is not updated by that command. The PIN is not changed. Reload the browser once so it picks up the new stamp.

IF THIS CLIENT USES THE DESKTOP TILL

A Worker deploy updates the browser. It does not update the till. A file already inside the program wins over the screen cache. Repeat Section 17 when the desktop program changed or when a screen in `stockridge\public` changed. Pull each repository in its own folder, print both short hashes, stage, and build `dist\win-unpacked\StockRidge.exe`. Do not open `dist\StockRidge-0.1.0-win-x64.exe`. Repeat Section 18 only when the Android program itself changed. The till and the APK still have to be aimed at this client's Worker. A new build does not move an old till off the wrong host by itself. Do not deploy the Worker merely to refresh the till.

20 Useful Checks

```
# Who is Wrangler signed in as?
npx wrangler whoami

# Which environment does this folder declare?
Select-String -Path .\worker\wrangler.toml -Pattern "name =|database_name"
Select-String -Path .\worker\wrangler.toml -Pattern "TURSO_DATABASE_URL|^\[env\."

# Applied D1 migrations, only if D1 is the live store
$Db = "northline-stockridge-db"
$Cfg = "worker/wrangler.toml"
npx wrangler d1 migrations list $Db --remote --config $Cfg

# Turso: migrations recorded, token not printed
node tools/turso-admin.js migrate

# Health — JSON, not HTML
Invoke-RestMethod "$WorkerUrl/api/health"
Invoke-RestMethod "$WorkerUrl/api/health/ready"

# Administrator exists. The query returns no PIN and no hash.
$Sql = "SELECT username, role, is_active FROM users WHERE role = 'ADMIN'"
npx wrangler d1 execute $Db --remote --config $Cfg --command $Sql
turso db shell northline-stockridge $Sql

# Greenbook rows: thousands, not zero. Not proof of a pharmacy.
turso db shell northline-stockridge "SELECT COUNT(*) AS n FROM nafdac_catalog;"

# Git has no accidental secrets waiting
git status
git check-ignore .env.deploy
git check-ignore .data/d1-seed.sql
```

From this client's `destop-stockridge` folder, after a till build:

```
# Origin baked into the last build. Must be this client's Worker. No token.
Get-Content .\build\desktop-config.json

# The program you hand over. The portable exe is not this build.
Get-Item .\dist\win-unpacked\StockRidge.exe
Get-Item .\dist\StockRidge-0.1.0-win-x64.exe -ErrorAction SilentlyContinue
```

NOTE

On a D1-only client, use the Wrangler form of the administrator and catalogue queries, and do not call `turso db shell`. On a Turso client, a D1 query can look healthy while the Worker is reading a different database. Query the store named in the register.

21 Stop Immediately If Any of These Happen

DANGER SIGNS

- The Worker name is still `stockridge` or `sample` and this is a new client.
- The database name or id is one of the downloaded demonstration ids in Section 8.
- `TURSO_DATABASE_URL` still contains `stockridge-10client`.
- The folder, the Git remote, and the Cloudflare account in `npx wrangler whoami` are not the same row of the register.
- `git status` shows `.env.deploy`, `.env`, `.data\d1-seed.sql`, a backup SQL file, or a database file.
- The deploy summary says production, and you meant sample — or the reverse.
- `/api/health` returns HTML instead of JSON.
- `/api/diagnose` answers as another application. Another Worker owns this name.
- You are about to pass `--reset-pin`, run `format-admin-only`, or run `copy-from-d1` against a database that has real sales.
- The summary printed a PIN marked unverified, and you are about to send that number to the client.
- You are about to add a `[limits]` block to raise the Free-plan subrequest cap. Free cannot raise it, and a CPU-limit block fails the upload. Turso with one socket is the path this project uses instead.
- `STOCKRIDGE_ORIGIN` was unset, or `desktop-config.json` names another client's Worker, or the demonstration host, and this is not that demonstration.
- You are about to open or hand over `dist\StockRidge-0.1.0-win-x64.exe`. That portable file is not the unpacked build.
- You ran `git pull` or `npm run archive` in a parent folder, or the path contains a space and the build was started through `npm.cmd`.
- The short hashes were not printed in both folders before the build, so you cannot tell which program the shop will open.
- `destop-stockridge` contains `.env.deploy`, a Turso token, a Cloudflare token, or a PIN.
- `STOCKRIDGE_ROOT` points at a different client's `stockridge` folder.
- The Electron window still shows Download or Install. That program is older than the hide, or it is the portable exe. Build the unpacked program again. Do not deploy the website to remove it.
- A client's APK still names `sample.stockridge.workers.dev`, or the office host, in `strings.xml`.

Correct the configuration first. Do not “try it and see” with a client's books, or with the demonstration books.

CORE RULE, AGAIN

one client folder → one Git remote/SSH alias → one Cloudflare account → one Worker → one live database. If a step mixed two of those, it is not finished.